

SCOPE: Array-Native Nonlinear Acceleration for LLM Inference via Shape-Constrained Neural Approximation

MICRO 2026 Submission #614 – Confidential Draft – Do NOT Distribute!!

Abstract

Nonlinear operators such as Softmax, activation functions, and normalization layers increasingly bottleneck large language model (LLM) inference on GEMM-centric accelerators. The bottleneck becomes more severe under quantization because the linear path continues to benefit from low-precision systolic-array execution while nonlinear functions are still typically handled by separate scalar, vector, or special-function units. Existing solutions still struggle to provide broad operator support, high approximation fidelity, and hardware-efficient, scalable execution at the same time. We present **SCOPE**, a software-hardware co-design that turns nonlinear evaluation into an array-native capability. **SCOPE** proposes a shape-constrained neural approximator (SCNA), a compact one-hidden-layer ReLU network that captures structural priors of nonlinear functions in a unified and hardware-friendly form. **SCOPE** also introduces the array-native nonlinear execution that maps SCNA directly onto a lightly extended systolic array, and a scale-fused quantization dataflow that keeps the nonlinear path on the same low-precision foundation as surrounding quantized GEMMs. Across 11 nonlinear functions and five representative LLMs, **SCOPE** reduces geomean approximation MSE by 361× over NN-LUT and 14.9× over T-LUT, maintains BF16/INT8 perplexity and zero-shot accuracy with limited additional degradation, achieves up to 2.81× long-context attention speedup over FlashAttention on TPUs, achieves up to 2.69× full-prefill speedup under INT8, and lowers normalized area and power by 12.8× and 9.5×, respectively, compared with prior nonlinear accelerators. The implementation is available at <https://anonymous.4open.science/r/SCOPE-Nonlinear-DD23/>.

1 Introduction

Transformer-based large language models (LLMs) now deliver state-of-the-art results across vision and language tasks [6, 8, 13, 15, 33]. Their inference is dominated by general matrix multiplication (GEMM), so modern LLM accelerators are increasingly built around systolic arrays as the primary compute engines [28, 29, 32]. To further improve throughput and efficiency, modern deployment increasingly adopts low-precision formats such as INT8 for weights and activations [12, 16, 36, 53]. Alongside GEMM, LLM inference also relies on nonlinear layers such as Softmax [17], activation functions such as GeLU and SwiGLU [23, 48], and normalization layers such as LayerNorm and RMSNorm [4, 58]. These layers depend on nonlinear primitives such as Exp, Sigmoid, Tanh, and Rsqrt.

These nonlinear operators are emerging as a major bottleneck in modern LLM inference. Unlike GEMM, they do not map naturally to GEMM-centric hardware and are therefore typically executed on separate scalar, vector, or special function unit (SFU)-style pipelines, as illustrated in Figure 1a. Because nonlinear functions are more

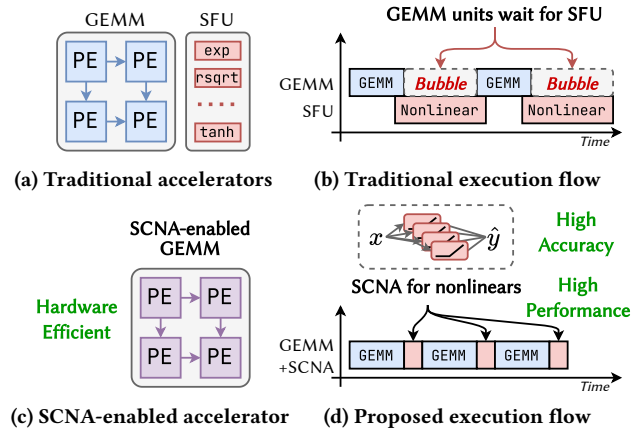


Figure 1: SCOPE enables array-native nonlinear execution with shape-constrained neural approximators (SCNA).

complex and less regular than GEMM, SFU-based implementations incur nontrivial area and power overhead and do not scale with the main systolic array [20, 43, 56]. As GEMM execution continues to accelerate, especially under quantization, nonlinear processing accounts for an increasingly visible share of end-to-end execution cost and becomes a system-level bottleneck, as shown in Figure 1b. Recent systems trends support this view: FlashAttention-4 redirects exponential work toward FMA resources to reduce MUFU/SFU pressure, while Blackwell Ultra/B300 increases SFU throughput to shrink softmax stalls [34, 56].

Prior work reduces nonlinear cost from several directions, but a key gap remains. Lookup-table (LUT), piecewise-linear (PWL), and learned-surrogate approaches can accelerate target functions, but high accuracy often comes with growing table capacity, finer segmentation, or more complex function-specific control [9, 14, 42, 54, 55]. Programmable nonlinear engines broaden operator coverage, yet they still treat nonlinear computation as a separate execution domain with its own mapping and control overhead [20, 43]. Array-fused designs move nonlinear processing closer to the main compute substrate, but many remain specialized or still rely on substantial auxiliary support such as LUTs, buffering, routing, or mixed-precision handling [35, 39, 49]. Consequently, existing approaches still fall short of making nonlinear execution broad, accurate, and natively scalable with the main array substrate.

We take a different approach. Instead of treating nonlinear functions as exceptional operations handled by separate hardware, we turn nonlinear evaluation into a capability of the systolic array itself. Our key observation is that many nonlinear operators in LLM inference exhibit strong shared structure over their practical input domains, and can therefore be represented by a compact approximation form built from affine transforms and ReLUs. This form is especially attractive for array-based execution: affine transforms

are natively supported by GEMM-style systolic computation, and ReLU requires only lightweight element-wise support. As shown in Figure 1c–1d, this allows nonlinear functions to be executed on the same scalable compute substrate as GEMMs, making nonlinear execution part of the main array datapath.

Based on this observation, we propose **SCOPE**, a co-design for nonlinear acceleration in LLM inference. **SCOPE** introduces a *shape-constrained neuron approximator* (SCNA), a compact one-hidden-layer ReLU network that encodes structural priors such as monotonicity and convexity to enable compact and robust unified approximation. **SCOPE** further introduces a lightweight systolic-array extension that executes SCNA directly on array MAC resources, making nonlinear evaluation a native capability of the main compute substrate. A scale-fused quantization dataflow then integrates nonlinear execution into the low-precision inference pipeline, reducing conversion overhead across operator boundaries. In summary, this paper makes the following contributions:

- We propose **SCNA**, a compact and unified nonlinear approximation method that encodes shared structural priors of nonlinear operators into a one-hidden-layer ReLU network. It improves accuracy and robustness with only a small number of neurons.
- We introduce an **array-native nonlinear execution** mechanism that maps SCNA directly onto a lightly extended systolic array, enabling scalable nonlinear execution with GEMM units.
- We present a **scale-fused quantization dataflow** that absorbs quantization scales into offline SCNA parameterization, keeping the nonlinear path aligned with the same low-precision GEMMs.
- Our experiments show that **SCOPE** reduces approximation MSE by **361× over NN-LUT** and **14.9× over T-LUT**, **maintains BF16/INT8 LLM inference quality with limited additional degradation across five evaluated models**, achieves up to **2.81× attention-kernel speedup over FlashAttention**, **achieves up to 2.69× full-prefill speedup under INT8**, and lowers normalized area and power by **12.8× and 9.5×**, respectively, versus prior nonlinear accelerators.

2 Background and Motivation

2.1 LLM Inference and Quantization

Large language models (LLMs) are built from repeated transformer blocks [52] that combine dense matrix multiplications, namely general matrix multiplications (GEMMs), with nonlinear layers, as illustrated in Figure 2. Within each block, self-attention projects the input into query, key, and value tensors (Q , K , and V), while the feed-forward network applies two additional linear transformations. These stages are dominated by GEMMs.

The main nonlinear layers include Softmax [17] in attention [31]. The feed-forward network uses GeLU [23] or SwiGLU [48], and normalization uses LayerNorm [4] or RMSNorm [58]. These layers invoke nonlinear operators such as Exp, Erf, Sigmoid, and Rsqrt, as well as reductions and elementwise multiplication. They therefore do not naturally map to GEMM-centric hardware. Scaled dot-product attention [52] provides a representative example:

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V, \quad \text{softmax}(S)_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}.$$

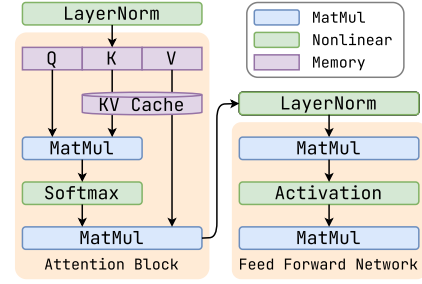


Figure 2: Typical structure of one transformer layer.

Here, QK^T and the final multiplication with V map naturally to GEMM accelerators, whereas Softmax introduces nonlinear computation that is much less compatible with systolic execution [32].

To reduce model size and memory traffic, modern LLM inference increasingly adopts low-precision formats such as INT8 or FP8 for weights and activations [12, 16, 36, 53]. In quantized inference, a real-valued tensor X is typically represented by a low-precision tensor $\hat{X}$ together with a scale factor s_X :

$$\hat{X} = Q_B(X; s_X), \quad X \approx s_X \hat{X},$$

where $Q_B(\cdot)$ denotes quantization to a target format B . This representation strongly benefits GEMMs because modern accelerators typically provide much higher GEMM throughput at low precision. Nonlinear operators, in contrast, are often more numerically sensitive and therefore frequently remain in higher precision. Practical quantized LLM inference therefore commonly involves mixed-precision execution across linear and nonlinear stages, together with explicit scale handling around nonlinear layers [26, 37].

2.2 Nonlinear Execution Bottlenecks

The growing cost of nonlinear operators stems from execution asymmetry in modern LLM accelerators. GEMMs map efficiently to systolic-array-style tensor engines with regular dataflow and scalable throughput, and they benefit further from low-precision formats such as INT8 and FP8. Nonlinear operators, in contrast, are typically executed on narrower scalar, vector, or special function unit (SFU) pipelines, whose throughput scales much less effectively with the main GEMM substrate because they require function-specific logic and less regular execution [28, 29].

We use FlashAttention [11], a widely used IO-aware exact attention algorithm, as a representative example. The AWS Neuron Kernel Interface (NKI) attention_cte kernel exposes a similar optimized execution flow for long-sequence attention [3]. Figures 3 and 4 summarize the timeline and utilization breakdown on AWS NeuronCore-v2. In this kernel, score computation ($S = QK^T$) and output multiplication ($O = PV$) are mapped to the tensor engine, whereas the exponential stage inside Softmax ($P = \exp(S)$) is executed on the narrower SFU-backed scalar path. The critical path is therefore determined not only by matrix multiplication, but also by nonlinear processing. This imbalance is also reflected in Figure 4a: even under FP16, tensor-engine utilization remains below 50%, while the SFU or scalar path shows much higher activity because of the exponential stage.

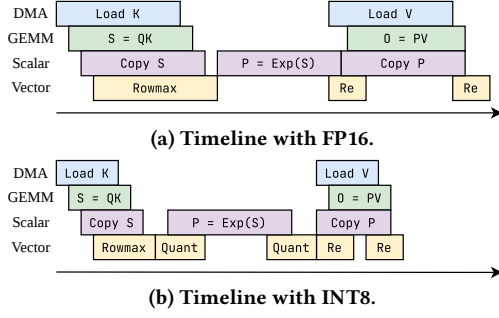


Figure 3: Kernel-level timeline when executing FlashAttention on AWS NeuronCore-v2.

The imbalance becomes even more severe under quantization. Although GEMMs become faster in INT8, nonlinear operators often remain in higher precision, introducing additional dequantization, quantization, and scaling around nonlinear stages. In the INT8 timeline of Figure 3b, these mixed-precision boundaries add extra work to the non-GEMM path. Figure 4b correspondingly shows that tensor-engine utilization drops to around 20%, while vector activity rises substantially because of frequent format conversion and quantization. Quantization therefore accelerates the linear path more than the nonlinear path, further widening the execution imbalance.

The key issue is throughput matching, not the absence of pipelining. A dedicated nonlinear pipeline can hide latency when its consumption rate matches the tensor engine’s production rate, but stalls reappear once GEMM throughput grows faster than SFU/vector throughput. This pressure is especially visible in low-precision inference, where INT8 or FP8 GEMM throughput increases while transcendental and scale-handling paths often remain narrower.

2.3 Prior Nonlinear Acceleration Solutions

Prior hardware proposals address the nonlinear bottleneck from several directions, but none simultaneously provides broad operator support, low hardware overhead, clean scaling with the main GEMM engine, and compatibility with low-precision execution.

Lookup- and PWL-based approximation. One line of work replaces function-specific nonlinear units with more general approximation engines based on lookup tables or piecewise-linear (PWL) approximation [9, 14, 42, 54, 55]. T-LUT [54] reduces storage redundancy through a twofold lookup-table structure, while NN-LUT [55] further reduces the effective table size. These designs, however, still rely on breakpoint management, segment selection, and comparison logic. As approximation accuracy increases, table size, control complexity, and hardware overhead also grow.

Programmable nonlinear engines. Another direction improves flexibility by introducing programmable nonlinear substrates. PICACHU [43] proposes a plug-in coarse-grained reconfigurable array (CGRA) for LLM nonlinear operators, while Tandem [20] provides an on-chip companion processor for non-GEMM layers. These designs broaden operator coverage, but they do so through a separate nonlinear execution path with its own mapping flow and software stack. They therefore improve flexibility without removing the fundamental scaling mismatch between the nonlinear path and the much faster GEMM path.

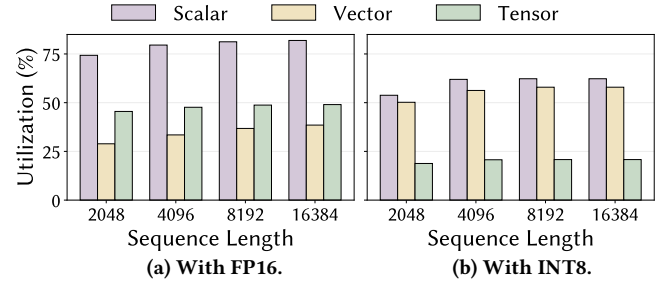


Figure 4: Utilization of different components when executing FlashAttention kernel on AWS NeuronCore-v2.

Array-fused nonlinear execution. A third direction attempts to fuse nonlinear execution into the systolic array itself. SystolicAttention [35], FuseMax [39], and OneSA [49] all move part of nonlinear execution into the array, reducing reliance on separate side pipelines. This direction is promising, but existing designs still inherit one of two limitations: they either target a narrow operator family, especially Exp and Softmax, or require nontrivial auxiliary support such as lookup structures, buffering, or routing logic.

Common limitations under modern LLM inference. Prior work shares three key limitations. First, many designs support only a narrow set of operators or depend on function-specific approximation structures. Second, their hardware cost does not scale cleanly with throughput due to expanding lookup/control overhead or reliance on a separate nonlinear engine. Third, few fully address low-precision nonlinear execution, and many still fall back to FP16/FP32 with extra format-conversion overhead. As a result, prior solutions do not yet provide a unified nonlinear substrate that is general, hardware-efficient, array-scalable, and compatible with modern low-precision LLM inference.

3 SCOPE Overview

3.1 Opportunity for Neural Approximation

We observe that neural approximation offers a promising path to unified nonlinear acceleration [22, 25, 38, 41]. Instead of provisioning separate function-specific hardware, a shared neural approximator can represent diverse nonlinear operators using a common form built from simple arithmetic primitives, enabling broad operator support on a unified substrate rather than multiple dedicated units or side pipelines. This is particularly attractive for LLM inference, where many target operators, including Softmax-related exponentials, activation functions, and normalization primitives, are structurally similar over bounded input domains. Our key insight is that these shared structural properties make them amenable to a common neural approximation form whose dominant computation aligns well with GEMM-style execution, consisting primarily of multiply-accumulate operations, bias addition, and simple activation. As a result, most nonlinear execution can be absorbed into the primary array datapath, enabling a more compact design and tighter integration with the main accelerator substrate.

3.2 Design Challenges

While neural approximation is a promising foundation for unified nonlinear acceleration, directly applying it to modern LLM inference remains non-trivial. We identify three key challenges.

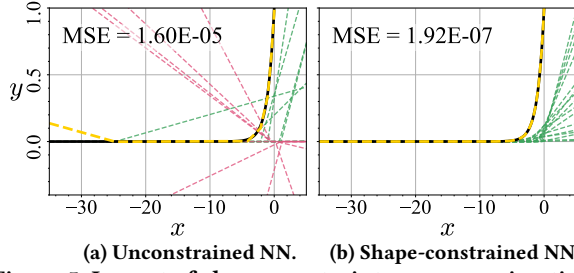


Figure 5: Impact of shape constraints on approximation.

Challenge 1: Structured rather than unconstrained approximation. Many LLM nonlinear operators share strong shape priors over their practical domains. Unconstrained neural approximation, however, fails to exploit this structure and often introduces redundant or misaligned basis components that waste model capacity and hurt robustness. The challenge is to derive a compact unified approximation form that explicitly encodes these shared priors.

Challenge 2: Scalable execution on array-based accelerators. Even a compact approximation form is of limited value if it still requires irregular control, lookup-heavy support, or a separate nonlinear engine. The challenge is to execute nonlinear approximation on the same systolic substrate as GEMMs while preserving regularity, scalability, and low hardware overhead.

Challenge 3: End-to-end compatibility with low-precision inference. Quantized LLM inference accelerates the linear path, but nonlinear stages often remain in higher precision and introduce repeated format conversion and scale-handling overhead. The challenge is to integrate nonlinear execution into a quantization-friendly pipeline so that its benefit is preserved at system level.

3.3 Technique Overview

To address the three challenges above, **SCOPE** adopts three co-designed techniques that together make nonlinear execution compact across diverse operators, efficient on array-based accelerators, and compatible with low-precision inference.

Technique 1: Shape-constrained neural approximation. To support diverse nonlinear operators under a tight hardware budget, **SCOPE** proposes a shape-constrained neural approximator that embeds structural priors such as monotonicity, symmetry, and convexity into the approximation form. This improves approximation efficiency and enables a compact unified representation across attention, activation, and normalization operators.

Technique 2: Array-native nonlinear execution. To execute this approximation efficiently on array-based accelerators, **SCOPE** maps the neural approximation directly onto the systolic array with minimal architectural extension. By reusing simple arithmetic primitives that naturally fit the array datapath, it avoids LUT-heavy support and separate nonlinear side pipelines while preserving scalability with the main GEMM engine.

Technique 3: Scale-fused quantization dataflow. **SCOPE** integrates nonlinear execution into the quantized inference pipeline through a scale-fused low-precision dataflow. This dataflow avoids repeated dequantization and requantization across operator boundaries, reduces scale-handling overhead, and ensures that nonlinear acceleration translates into real end-to-end gains.

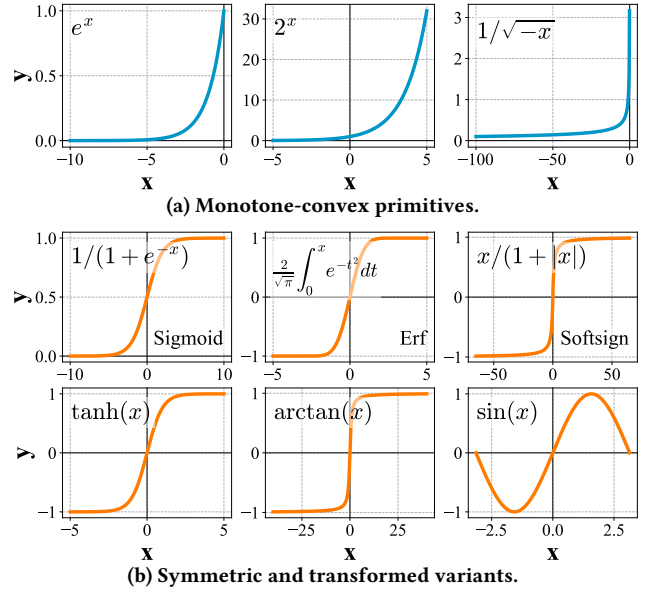


Figure 6: Structural families of nonlinear functions in LLMs.

4 Shape-Constrained Neural Approximators

4.1 Pitfalls of Unconstrained Approximation

Several prior works use single-hidden-layer ReLU networks to approximate nonlinear functions [50, 55]. Our analysis reveals an overlooked limitation of optimizing only output error: the internal structure of the learned approximation remains unconstrained. Even when the fitted output matches the target well within the training range, the approximation can still be composed of aligned and misaligned basis functions that partially cancel each other.

Figure 5 illustrates this phenomenon for Exp. Since Exp is monotone increasing and convex, its approximation should ideally be constructed from neurons with the same trend. Unconstrained training, however, can produce a mixture of aligned basis functions (green curves) and misaligned basis functions (red curves), whose cancellation is hidden in the final output. This hidden cancellation wastes model capacity, weakens extrapolation beyond the training range, and reduces robustness under low-bit quantization.

This observation motivates *Shape-Constrained Neural Approximators* (SCNA), which enforce shape consistency during training to remove such cancellation and improve both approximation quality and deployment robustness, as illustrated in Figure 5b.

4.2 Shared Shape Priors in Nonlinear Functions

Although nonlinear operators in LLM inference appear diverse, many of them can be reduced to a small set of geometric templates. Figure 6 summarizes the main structural families used by SCNA. One important template is the *monotone-convex* shape. Representative functions such as Exp, Exp2, and Rsqrt (after reflection) all exhibit this property on the negative half-axis.

Formally, a function f is *monotone-convex* on $x \in (-\infty, 0)$ if

$$\text{Monotone: } x \leq y \Rightarrow f(x) \leq f(y),$$

$$\text{Convex: } f(\alpha x_1 + (1 - \alpha)x_2) \leq \alpha f(x_1) + (1 - \alpha)f(x_2),$$

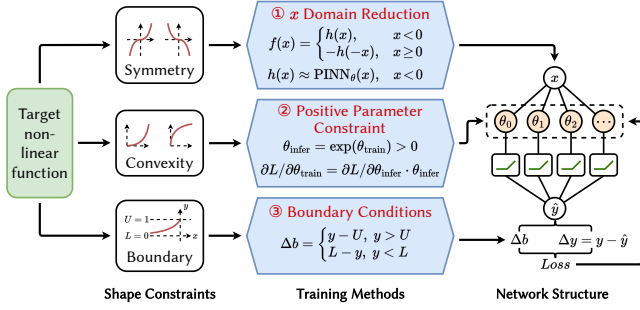


Figure 7: Domain reduction, parameter constraints, and boundary-aware training in SCNA.

where $\alpha \in [0, 1]$. Many important operators, such as Exp and Exp2, directly satisfy this property. Several other functions do not globally follow the monotone-convex template, but they can still be reduced to it through simple transformations. For example, Rsqrt, defined as $x^{-1/2}$, is convex and monotone decreasing on $x > 0$, and can be converted into the same canonical form by input reflection. Similarly, Tanh and Sigmoid exhibit central symmetry,

$$f(x) + f(-x) = a,$$

where $a = 0$ for Tanh and $a = 1$ for Sigmoid. Moreover, both functions are monotone increasing and convex on the negative half-axis. It is therefore sufficient to approximate only the branch $x \leq 0$ and recover the positive branch by symmetry. Periodic functions such as Sin can likewise be reduced to a canonical interval $(-\pi/2, 0)$ using periodicity and reflection, where Sin is also monotone-convex. For functions with negative values, we further apply a constant shift so that the approximated canonical branch remains nonnegative.

Overall, many nonlinear operators in LLM inference can be reduced to a small number of canonical branches, among which the monotone-convex family is the dominant one. This shared shape prior provides the foundation for *Shape-Constrained Neural Approximators* (SCNA) and enables more compact and robust approximation than unconstrained neural forms. The shape prior is a training strategy rather than a hardware restriction. The array datapath in Section 5 can execute any sum of affine-plus-ReLU terms, while the monotonicity, convexity, symmetry, reflection, and bound constraints make the representation compact and robust for the scalar continuous nonlinearities that dominate LLM attention, activation, and normalization paths.

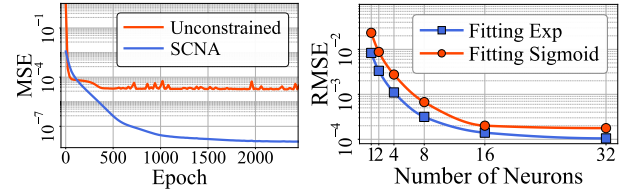
4.3 SCNA Formulation

This canonical *monotone-convex* structure maps naturally to a single-hidden-layer ReLU network:

$$g(x) = \sum_{i=1}^S k_i \cdot \text{ReLU}(w_i x + b_i), \quad (1)$$

where $\text{ReLU}(\cdot)$, the core ReLU primitive of our approximator, is itself *monotone-convex*, and is also efficient to implement in hardware.

$\text{ReLU}(x \cdot w_i + b_i)$ is the composition of an affine function $x \cdot w_i + b_i$ and $\text{ReLU}(\cdot)$. To make this branch increasing, the affine function must also be increasing, which requires $w_i \geq 0$. The positivity of k_i is likewise necessary to preserve the monotone-convex property. We therefore require $k_i, w_i \geq 0$. Moreover, the target domain $x \in (-\infty, 0)$ requires the hinge point of ReLU to be non-positive:



(a) Training loss comparison. (b) RMSE versus neuron count.

Figure 8: SCNA training and deployment recipe.

Table 1: SCNA training recipes.

Function	Property	Input Range	λ_{bound}
Exp	Monotone-convex	$(-256, 0)$	0.15
Exp2	Monotone-convex	$(-256, 0)$	0.15
Sigmoid	Symmetric at $(0, 0.5)$	$(-6.234, 6.234)$	0.10
Erff	Symmetric at $(0, 0)$	$(-2.469, 2.469)$	0.15
Rsqrt	Reflected at $x = 0$	$(0.1, 1024)$	0.10
Sin	Symmetric at $(0, 0)$	$(-\pi/2, \pi/2)$	0.30
Tanh	Symmetric at $(0, 0)$	$(-3.465, 3.465)$	0.20
Softsign	Symmetric at $(0, 0)$	$(-128, 128)$	0.15
Arctan	Symmetric at $(0, 0)$	$(-128, 128)$	0.15

$t_i = -b_i/w_i \leq 0$. Otherwise, the corresponding component contributes zero over the target range. This condition yields $b_i \geq 0$. As a result, parameter fusion can be utilized to further simplify the approximator in Equation 1:

$$\begin{aligned} g(x) &= \sum_{i=1}^S \text{ReLU}((k_i w_i) \cdot x + (k_i b_i)) \\ &= \sum_{i=1}^S \text{ReLU}(w_i^* \cdot x + b_i^*) \end{aligned}$$

with $w_i^* \equiv k_i w_i \geq 0$ and $b_i^* \equiv k_i b_i \geq 0$. On any compact interval, this parameterization therefore provides a natural approximator for monotone-convex targets. We define SCNA as a single-hidden-layer ReLU-based network whose parameters are all non-negative:

$$\text{SCNA}_{\theta}(x) = \sum_{i=1}^S \text{ReLU}(w_i \cdot x + b_i) \quad (2)$$

where $\theta = \{w_i, b_i\}_{i=1}^S$ and $w_i \geq 0, b_i \geq 0$ for every i .

4.4 SCNA Training from Shared Shape Priors

To deliver practical training recipes for SCNA, we define three components: how to prepare input domains and output targets that align with the shape priors, how to encode the nonnegativity constraints during training, and how to define the training loss so that it reflects function bounds. Figure 7 summarizes this recipe. We first reduce each target operator to a canonical branch with consistent monotone-convex structure by using symmetry or reflection. We then use exponential reparameterization to enforce the nonnegativity constraints that encode the monotone-convex shape prior. Finally, we add a boundary-aware loss term on top of the mean squared error (MSE) objective to regularize training within the valid output range.

Canonical branch reduction. We first transform each target function into a canonical branch that matches the monotone-convex structure exploited by SCNA.

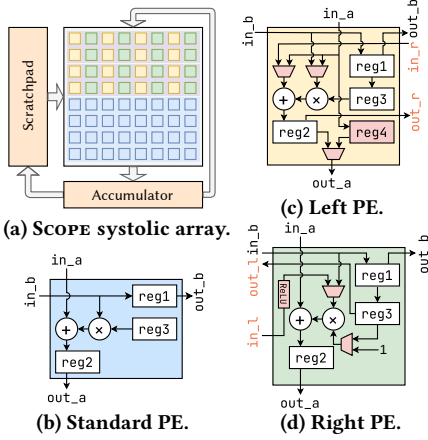


Figure 9: Microarchitecture for array-native SCNA execution.

For centrally symmetric targets, including Sigmoid, Erf, Tanh, and Softsign, we learn only the negative-half branch $h(x)$:

$$f(x) = \begin{cases} h(x), & x < 0, \\ a/2, & x = 0, \\ a - h(-x), & x > 0. \end{cases} \quad (3)$$

where $f(0) = a/2$, which equals 0 for most target functions; for Sigmoid, $a = 1$.

For decreasing convex functions defined on $(0, \infty)$ such as $1/x$, $1/\sqrt{x}$, we reflect the function about $x = 0$ and train $h(x)$ on the negative half-axis: $f(x) = h(-x)$. If $f(x)$ is decreasing and convex on $(0, \infty)$, then $f(-x)$ is increasing and convex on $(-\infty, 0)$. After these transformations, SCNA trains on a canonical branch with shape priors.

Positive Reparameterization. Once the target is reduced to a canonical monotone-convex branch, training must preserve the nonnegativity constraints of SCNA. To enforce $w_i, b_i \geq 0$, a trivial solution is to hard clip negative values after each optimization step:

$$w_{\text{new}} \leftarrow \max(w_{\text{opt}}, \epsilon), \quad \epsilon > 0$$

However, this approach discards the gradient information from the optimizer and hurts convergence smoothness, leading to suboptimal approximation quality [1, 24]. Instead, we use *positive reparameterization* to replace each constrained parameter by the exponential of an unconstrained raw parameter [45, 47]:

$$\begin{aligned} \text{SCNA}_\theta(x) &= \sum_{i=1}^S \text{ReLU}(w_i \cdot x + b_i) \\ &= \sum_{i=1}^S \text{ReLU}(e^{w_{i,\text{raw}}} \cdot x + e^{b_{i,\text{raw}}}) \end{aligned} \quad (4)$$

where $w_{i,\text{raw}}, b_{i,\text{raw}} \in \mathbb{R}$. This transformation guarantees nonnegativity while preserving smooth gradients for optimization. We therefore train the model on the raw parameters $\theta_{\text{raw}} = \{w_{i,\text{raw}}, b_{i,\text{raw}}\}_{i=1}^S$, and deploy it with the effective positive parameters $\theta = \{w_i, b_i\}_{i=1}^S$ with $w_i = e^{w_{i,\text{raw}}}$, $b_i = e^{b_{i,\text{raw}}}$.

Boundary-aware training. Many target operators are bounded or partially bounded, and these output limits are part of their shape prior. A pure MSE objective does not encode such constraints. We therefore use $L_{\text{total}} = L_{\text{MSE}} + \lambda_{\text{bound}} L_{\text{bound}}$, where $L_{\text{bound}} =$

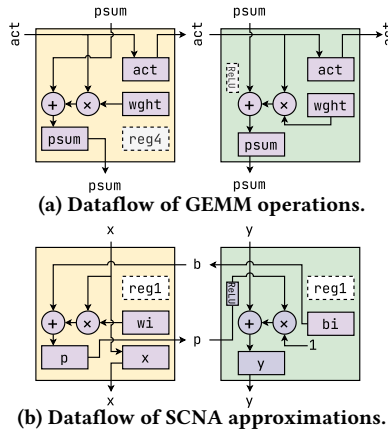


Figure 10: Dataflow for array-native execution of SCNA.

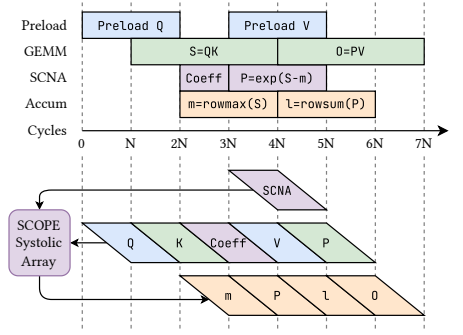


Figure 11: Cycle-level dataflow of fused FlashAttention.

$\max(0, \hat{y}_m - y_{\max})^2 + \max(0, y_{\min} - \hat{y}_m)^2$. $\hat{y}$ is the approximation result, and $[y_{\min}, y_{\max}]$ is the valid output interval.

Figure 8a shows that, on Exp, the constrained formulation converges faster and more smoothly than an unconstrained neural baseline while also achieving substantially lower final error. Figure 8b further highlights the neuron efficiency of SCNA by plotting final root mean squared error (RMSE) against neuron count. The RMSE drops markedly as the neuron count increases from 4 to 16, but decreases only marginally beyond 16, indicating that 16 neurons capture most standalone approximation benefit for the target functions. Table 1 summarizes the full deployment setup.

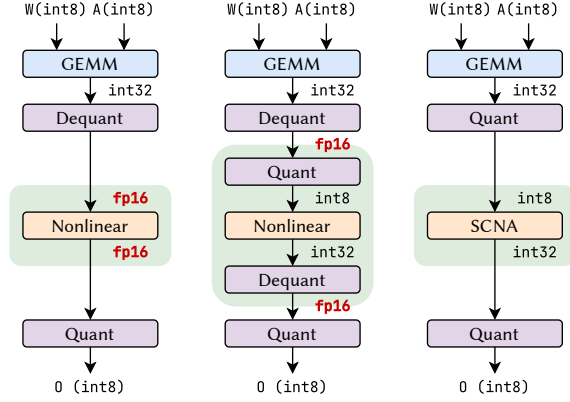
5 Array-Native Execution of SCNA

5.1 Systolic Array Extension

SCNA consists of repeated affine transformations followed by ReLU accumulation: $y' = y + \text{ReLU}(w_i x + b_i)$. This structure matches the native computation pattern of systolic arrays, which already provide multiply-accumulate (MAC) datapaths and regular nearest-neighbor dataflow. We therefore realize SCNA directly on the systolic substrate through a lightweight extension rather than through a separate nonlinear engine.

Architecture Overview. As shown in Figure 9a, the overall organization remains identical to a conventional systolic-array accelerator: a central two-dimensional processing-element (PE) array is coupled with a scratchpad for weights and activations, and an accumulator (ACC) for near-memory accumulation and post-processing. During standard GEMM execution, weights are preloaded from the scratchpad into the array, activations are streamed through the array, and output partial sums are accumulated along the reduction dimension in the ACC. The ACC can then either write results back to the scratchpad or restream them into the array for subsequent computation. Our goal is to preserve this execution model while adding scalable support for SCNA evaluation.

Lightweight PE-pair extension. The key modification is a heterogeneous PE organization, illustrated in Figure 9a. We use three PE variants: a standard PE (blue, Figure 9b), a left PE (yellow, Figure 9c), and a right PE (green, Figure 9d). All three variants preserve



(a) Conventional (b) Naive quantization (c) SCNA scale fusion

Figure 12: Nonlinear evaluation for quantized execution.

the preload and GEMM semantics of a conventional systolic array with identical latency. Compared with a standard PE, the extension is deliberately lightweight: the left PE adds only one extra register `reg4` with input-element width, the right PE adds only one small ReLU unit, and the new links are strictly local to adjacent PE. The added hardware cost and timing impact therefore remain modest.

5.2 Unified Execution of GEMM and SCNA

Our systolic extension supports both standard GEMM operations and SCNA evaluation. The GEMM dataflow and latency remain identical to the base array, while the SCNA dataflow leverages the new PE-pair structure to realize high-throughput nonlinear approximation. SCNA execution can also overlap with the next GEMM tile preload because it uses the vertical stream and local PE-pair state, allowing linear and nonlinear phases to switch without adding a separate nonlinear pipeline stage.

GEMM execution. Figure 10a shows the dataflow when the extended PEs execute ordinary GEMM operations. In this mode, the behavior is identical to that of a standard PE: $p'_{\text{sum}} = p_{\text{sum}} + w \times a$, where the activation a propagates horizontally, the partial sum p_{sum} propagates vertically, and the weight w is preloaded locally.

SCNA execution. The key difference appears in SCNA mode, shown in Figure 10b. A left-right PE pair jointly executes one SCNA neuron. The left PE computes the affine term $p = x \cdot w_i + b_i$, where x flows vertically and is staged by `reg4`, w_i is stored locally in the left PE, and b_i is stored in the adjacent right PE. The intermediate result p is then sent to the right PE, which computes $y' = y + \text{ReLU}(p)$, where y flows vertically as the running accumulated output. The addition is implemented by reusing the existing MAC datapath, with one multiplier input fixed to 1. A single adjacent PE pair therefore evaluates one SCNA neuron with one-element-per-cycle throughput: $y' = y + \text{ReLU}(w_i x + b_i)$.

Overlapping with weight preload. SCNA execution uses only the vertical dataflow and a small set of registers within the PE pair, leaving the horizontal dataflow and `reg1` free, as shown in Figure 10b. This permits direct overlap with weight preloading, which requires only data propagation and does not occupy MAC units. The overlap is critical for achieving consistent high throughput in kernels with rapid linear–nonlinear switching, such as FlashAttention, especially for systolic arrays with large N where preload

latency is significant. Additionally, the PE provides a mode to copy the preloaded weights from `reg1` to `reg3`, so the GEMM dataflow will not be affected.

Throughput analysis. For an SCNA with S neurons, S PE pairs are vertically stacked to evaluate the full approximator. On an $N \times N$ systolic array, one vertical strip of size $N \times S$ contains $N/2$ such SCNA stacks, providing $N/2$ elements-per-cycle throughput with latency $S + 1$. Since a conventional systolic array typically accepts N elements per cycle, at most two SCNA strips, occupying $N \times 2S$ PEs in total, need to be converted, while all remaining PEs stay in the standard GEMM configuration. This organization preserves the scalability of the main array while providing high-throughput nonlinear evaluation without a separate side pipeline.

FlashAttention dataflow. Our FlashAttention [11] mapping is designed so that Exp never becomes a standalone stage, as shown in Figure 11. Instead, once partial score outputs from the first GEMM QK^T begin to emerge at cycle $2N$, ACC starts the row-wise max reduction, and during cycles $3N \sim 5N$, as SA drains the score tile S , ACC immediately restreams it to SA for SCNA-based $\exp(S - \hat{m})$. This Exp phase overlaps with data movement that is already required after the first GEMM and is further hidden by the preload of the next V tile for the second GEMM. Starting at $4N$, the exponentiated tile P streams out, enabling the row-wise sum reduction and the startup of PV to proceed in parallel. As a result, Exp is overlapped precisely in the drain/restream window between the two GEMMs, removing the main serialization point of online Softmax and reducing the steady-state schedule to $5N + O(1)$ cycles.

6 Scale-Fused Quantization Dataflow

Prior low-precision nonlinear acceleration reduces the cost of nonlinear evaluation, but it still leaves an explicit boundary between quantized computation and real-valued scale semantics. Our scale-fused design removes this boundary by folding scale handling directly into the nonlinear datapath. Figure 12 compares the three dataflows considered in this section.

6.1 Mixed-Precision Boundary in Quantized LLM Inference

Figure 12a illustrates conventional quantized LLM inference. Low-bit execution is used for GEMM, but the following nonlinear stage typically restores the quantized result to full precision before evaluation [26, 37]. This dequantize–nonlinear–requantize sequence introduces extra conversion overhead and prevents nonlinear throughput from scaling with the low-bit GEMM datapath.

Figure 12b shows a stronger baseline in which the nonlinear stage also operates on low-precision data [30]. Even this baseline does not remove the core obstacle, because scale propagation through a nonlinear operator is not straightforward. For a quantized GEMM with weights $\hat{W}$, activations $\hat{A}$, and scales s_W, s_A , linearity gives

$$WA = (s_W s_A)(\hat{W} \hat{A}).$$

The scale therefore passes through GEMM directly, yielding output scale $s_O = s_W s_A$ and quantized output $\hat{O} = \hat{W} \hat{A}$. A nonlinear function, in contrast, does not in general admit such a direct scale mapping. Translating the input scale into the correct output scale is operator-dependent and nontrivial. This is why a dequantization

step is typically required to recover the unscaled real-valued input before applying the nonlinear function with its function-specific input/output scales and then representing the result back in the quantized numerical domain. Consequently, fusing the Dequant and Quant steps, as illustrated in Figure 12c, is nontrivial: it must correctly handle the input scale of the nonlinear function, and this handling is usually specific to each nonlinear operator.

6.2 Fusing Quantization Scales into SCNA

Our key observation is that ReLU, despite being nonlinear, is positively homogeneous: $\text{ReLU}(sx) = s \text{ReLU}(x)$ for $s > 0$. Because SCNA is a sum of ReLU bases, it inherits a structured scale-mapping property that lets us propagate quantization scales through the nonlinear stage much like through GEMM. This property enables the optimized dataflow in Figure 12c, where the nonlinear stage operates directly on low-precision inputs without a floating-point dequantization boundary.

Let $\text{SCNA}_\theta(\cdot)$, with $\theta = \{w_i, b_i\}_{i=1}^S$, denote the SCNA defined in Equation 2. For any positive scale $s > 0$,

$$\begin{aligned} \text{SCNA}_\theta(sx) &= \sum_{i=1}^S \text{ReLU}(w_i sx + b_i) \\ &= s \sum_{i=1}^S \text{ReLU}(w_i x + b_i/s) \\ &= s \cdot \text{SCNA}_{\theta_s}(x), \quad \theta_s = \{w_i, b_i/s\}_{i=1}^S. \end{aligned} \quad (5)$$

Thus, SCNA preserves the input scale at the output, provided that the scale s is absorbed into a reparameterized basis set θ_s . Under static quantization, the required input scales are fixed after calibration, so this reparameterization can be performed once offline.

This gives the scale-fused dataflow in Figure 12c. The upstream quantized GEMM produces an INT32 accumulation $\hat{G} = \hat{W}\hat{A}$ with scale $s_G = s_W s_A$, which is fixed under static calibration. We then apply one requantization with scale s_P to obtain the INT8 SCNA input $\hat{P}$, whose effective scale is $s_W s_A s_P$. Instead of dequantizing $\hat{P}$, we fuse this scale into the SCNA parameters offline by replacing θ with $\theta_{s_W s_A s_P}$, that is, by dividing each bias b_i by $s_W s_A s_P$. The nonlinear stage can then operate directly on $\hat{P}$, producing an INT32 output $\hat{Q} = \text{SCNA}_{\theta_{s_W s_A s_P}}(\hat{P})$, while preserving the same effective scale $s_W s_A s_P$. If this result feeds the next quantized GEMM, we apply a final static requantization with scale s_Q , yielding output $\hat{O}$ with total scale $s_W s_A s_P s_Q$.

All on-chip intermediates on the nonlinear path therefore remain low precision. Only the accumulated real-valued scale is carried as metadata rather than materialized at the nonlinear boundary. Compared with the naive low-precision flow in Figure 12b, the key change is that the nonlinear boundary becomes a low-precision requantization rather than a floating-point dequantize–requantize boundary. Nonlinear execution is therefore aligned with the same low-bit datapath that already accelerates quantized GEMMs.

The preprocessing overhead is minimal. Static quantized deployment already calibrates layer scales and activation clipping ranges, and SCNA already performs offline fitting of its basis parameters. Scale fusion adds only a layer-local reparameterization of the SCNA biases using the fixed calibrated input scale. The benefit is therefore not just a cheaper nonlinear kernel, but a cleaner quantized dataflow that keeps the entire nonlinear path on the same low-precision execution substrate as the surrounding GEMMs.

Table 2: Devices used for end-to-end performance evaluation.

Metric		B200 [40]	AWSv4 [2]	TPUv6e [21]
Systolic Array	Count	592	8	2
	Size	16×16	128×128	256×256
Throughput (TOPS)	FP16 GEMM	2242.970	671.089	917.504
	INT8 GEMM	4485.939 (2×)	2684.355 (4×)	1835.008 (2×)
	Vector	70.093	9.830	7.168
	SFU / Exp	4.381	39.322	3.584
FP16 GEMM / Exp ratio		512	17	256
Memory	Capacity	180 GB	144 GiB	32 GB
	Bandwidth	8.000 TB/s	4.900 TB/s	1.600 TB/s

7 Evaluation

7.1 Experimental Setup

Function-level accuracy. We study a broad family of nonlinear functions covering the main nonlinear operators in LLMs. The set includes attention-path functions (Exp, Exp2, and Sigmoid), activation-path functions (Tanh, Softplus, GeLU, and the nonlinear components in SwiGLU), and normalization-path functions (Rsqrt and the elementwise nonlinear portions of RMSNorm and LayerNorm). This coverage allows us to evaluate the generality of SCNA across the key nonlinearities in modern LLM inference. We use PyTorch outputs as the floating-point reference and compare against classical approximation methods and prior PWL-based approximators. **We use 16 neurons by default unless otherwise specified.** For scalability analysis, we vary the neuron count and measure how the approximation error changes with model capacity.

Model-level accuracy. We evaluate five widely used open-source LLMs: OPT-6.7B, Llama-2-7B, Llama-3-8B, Qwen2.5-7B, and Qwen3-8B. For the quantization setting, we quantize weights, activations, and the KV cache to INT8 statically with SmoothQuant [53]. For activation calibration, we use The Pile [18] with 128 sequences of 2048 tokens each. We measure end-to-end model quality using perplexity on WikiText-2 and zero-shot accuracy on ARC-e [10], HellaSwag [57], PiQA [7], and Winogrande [46] with lm-eval [19].

End-to-end performance. We evaluate end-to-end LLM performance using an extended version of LLMCompass [59]. LLMCompass uses SCALE-Sim [44] for cycle-accurate systolic-array simulation and reports an average prefill error of 0.69%. We further implement the performance model of FlashAttention following FlashAttention-2 [11], and integrate our SCNA-enabled systolic array into the same simulator. The timing model of our design is further validated against register-transfer-level (RTL) simulation. **We use 8 neurons by default because it provides a good balance between accuracy and efficiency.** For hardware baselines, we select the latest representative generations of systolic-array-based accelerators, as listed in Table 2, and import their device specifications into the LLMCompass configuration. To evaluate our design, we replace the baseline systolic array in the simulator with the SCNA-enabled variant while keeping the rest of the system configuration unchanged.

Baselines. Our baselines include the PWL-based approximators LinearLUT [9], NN-LUT [55], and T-LUT [54], as well as array-fused nonlinear accelerators including SystolicAttention (denoted as FSA) [35], FuseMax [39], OneSA [49], and PICACHU [43]. **We also add a B300-style sensitivity point that doubles baseline SFU**

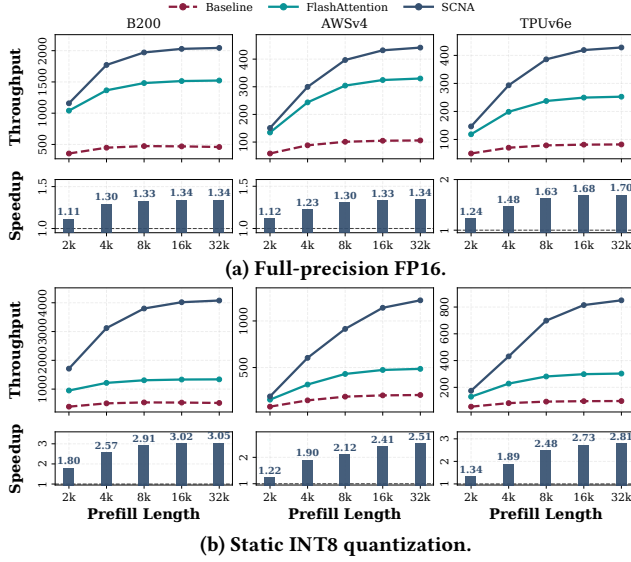


Figure 13: Prefill attention-kernel throughput comparison and the speedup over FlashAttention.

throughput following Blackwell Ultra [34] and discuss integer-only nonlinear flows represented by I-LLM [27] and IntAttention [60]. **Hardware implementation.** We implement the SCNA-enabled systolic array in Chisel [5] and synthesize the generated Verilog with Synopsys Design Compiler in TSMC 28nm at 1GHz. We adopt DesignWare IP [51] for floating-point arithmetic.

7.2 End-to-end LLM Performance

7.2.1 Attention-kernel Throughput. We evaluate attention-kernel prefill throughput over sequence lengths from 2K to 32K under both FP16 and INT8 execution. We report effective throughput in FLOPs/s as $(4Nd^2 + 2N^2d)/T$, where N is the sequence length, $d = 128$ is the head dimension, and T is kernel latency.

Figure 13 reports results on three representative accelerators, as listed in Table 2, together with the speedup of SCNA over a FlashAttention baseline. The results show that SCNA delivers the highest throughput in every evaluated setting, and its advantage consistently increases with context length. In FP16, SCNA achieves a 1.28× geomean speedup over FlashAttention, with gains reaching 1.34×–1.70× at 32K. The speedup curves are nearly monotonic with sequence length across all devices, which indicates that SCNA is especially beneficial in the long-context regime.

The benefit is even larger in INT8, which directly validates our motivation. Moving from FP16 to INT8 substantially increases GEMM throughput on modern accelerators, 2× in general and up to 4× on AWSv4, but the throughput of conventional nonlinear units does not scale proportionally. Nonlinearity therefore becomes a primary bottleneck. SCNA addresses this mismatch by providing an array-native nonlinear execution path whose throughput scales much more closely with the surrounding attention pipeline. As a result, the maximum speedup increases from 1.80× to 3.05× on B200, from 1.34× to 2.51× on AWSv4, and from 1.70× to 2.81× on TPUv6e when moving from FP16 to INT8. These results show that

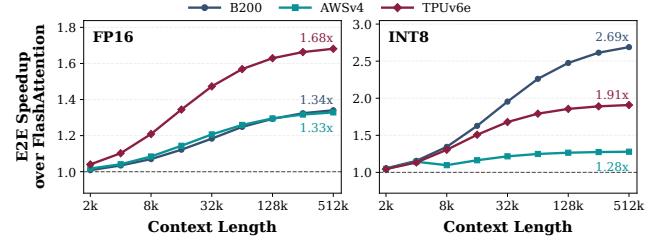


Figure 14: Full-prefill speedup over the FlashAttention baseline across long-context lengths.

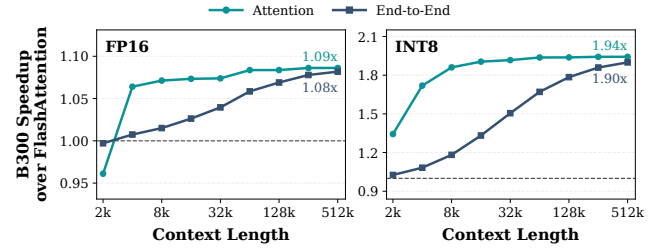


Figure 15: B300-style sensitivity study with increased baseline SFU/MUFU throughput.

Table 3: Prefill attention throughput (TFLOPS) comparison with integer-only nonlinear baselines.

Length	2K	4K	8K	16K
SCOPE	1130.86	1526.51	1672.82	1713.89
I-LLM	641.65 (-43.3%)	746.36 (-51.1%)	772.04 (-53.8%)	782.08 (-54.4%)
IntAttention	888.73 (-21.4%)	1093.30 (-28.4%)	1170.41 (-30.0%)	1104.60 (-35.5%)

SCOPE effectively removes the nonlinear bottleneck and translates that benefit into substantial LLM inference throughput gains.

7.2.2 Full-layer Prefill Throughput. We also model full transformer prefill latency, including attention, FFN, normalization, residual, and data-movement components. Figure 14 shows that attention-kernel gains translate into end-to-end speedup and grow with context length. At 32K context, full-prefill speedup is 1.207× on AWSv4, 1.183× on B200, and 1.473× on TPUv6e. At 512K context, the FP16 full-prefill speedup increases to 1.329×, 1.341×, and 1.681×, respectively; under INT8, the longest-context endpoints reach 1.28×, 2.69×, and 1.91× on AWSv4, B200, and TPUv6e. This Amdahl-style trend is expected because the accelerated attention fraction grows with context length.

7.2.3 Comparison to NVIDIA B300 with doubled SFU throughput. Figure 15 evaluates a B300/Blackwell Ultra-style baseline with doubled SFU/MUFU throughput. This stronger baseline reduces the exposed softmax stall, especially in FP16, but it does not eliminate the throughput mismatch. At the longest context, SCOPE still provides 1.09× attention and 1.08× full-prefill speedup in FP16, and 1.94× attention and 1.90× full-prefill speedup in INT8. The result supports our positioning: increasing dedicated SFU throughput is useful, while array-native nonlinear execution remains complementary and increasingly beneficial when low-precision GEMM throughput grows faster than the separate nonlinear path.

Table 4: Nonlinear function approximation accuracy.

Function	MSE (Mean Squared Error)								MAE (Mean Absolute Error)							
	Taylor	Frac-T	Interp	Frac-I	LinearLUT	NN-LUT	T-LUT	Ours	Taylor	Frac-T	Interp	Frac-I	LinearLUT	NN-LUT	T-LUT	Ours
Exp	INF	1.23E-8	3.60E-2	1.07E-9	4.41E-6	1.24E-6	1.24E-6	5.02E-9	INF	2.47E-6	5.85E-2	2.47E-6	1.40E-4	2.25E-4	1.95E-3	1.12E-5
Exp2	INF	1.75E-8	3.37E-2	1.55E-9	2.73E-6	1.28E-6	1.24E-6	1.70E-8	INF	8.48E-6	5.69E-2	3.57E-6	1.35E-4	2.00E-4	1.95E-3	2.38E-5
Sigmoid	1.77E-3	N/A	1.25E-4	N/A	5.59E-8	2.36E-6	1.23E-6	6.78E-8	2.61E-2	N/A	8.26E-3	N/A	1.23E-4	1.20E-3	1.95E-3	2.18E-4
Softsign	N/A	N/A	5.24E-2	N/A	3.58E-6	3.45E-4	1.27E-6	2.11E-7	N/A	N/A	1.02E-1	N/A	2.50E-4	1.39E-2	1.95E-3	2.97E-4
Softplus	5.98E-2	N/A	1.75E-4	N/A	7.35E-7	3.89E-6	1.26E-6	5.93E-8	1.45E-1	N/A	1.02E-2	N/A	5.02E-4	1.63E-3	1.95E-3	1.55E-4
Tanh	4.64E-2	N/A	4.55E-5	N/A	6.50E-8	9.68E-6	6.52E-7	1.61E-7	1.52E-1	N/A	4.57E-3	N/A	1.21E-4	2.58E-3	1.95E-3	3.34E-4
Arctan	N/A	N/A	1.40E-1	N/A	8.41E-6	2.25E-4	1.26E-6	3.04E-7	N/A	N/A	1.66E-1	N/A	3.92E-4	7.85E-3	1.95E-3	3.67E-4
Erf	N/A	N/A	3.74E-4	N/A	1.85E-7	8.38E-6	4.32E-7	1.30E-7	N/A	N/A	1.47E-2	N/A	2.46E-4	2.30E-3	1.95E-3	3.03E-4
Sin	5.23E-4	N/A	9.74E-5	N/A	1.30E-7	2.99E-3	N/A	6.47E-8	1.27E-2	N/A	8.15E-3	N/A	2.46E-4	5.47E-2	N/A	2.16E-4
Rsqrt	N/A	N/A	3.79E-1	N/A	8.05E-5	2.10E-2	N/A	1.22E-6	N/A	N/A	1.90E-1	N/A	5.40E-4	1.10E-1	N/A	7.41E-4
GeLU	1.65E-1	N/A	3.27E-5	N/A	6.10E-9	6.67E-6	1.19E-6	7.84E-8	2.41E-1	N/A	3.41E-3	N/A	3.63E-5	2.08E-3	1.95E-3	1.67E-4
Geomean	1.67E5×	N/A	2.38E4×	N/A	7.47×	361×	14.9×	1.00×	337×	N/A	146×	N/A	1.18×	20.5×	14.3×	1.00×

7.2.4 Comparison to prior fully integer nonlinear flows. Table 3 reports the prefill attention throughput under different sequence lengths for two pure-integer baselines, I-LLM and IntAttention. We reimplemented their softmax algorithms as optimized Triton kernels, benchmarked them against PyTorch native softmax on an H100 GPU, and incorporated the measured throughput ratios into our simulator to estimate attention kernel latency. **SCOPE** consistently achieves the highest throughput, increasing from 1130.86 TFLOPS at 2K to 1713.89 TFLOPS at 16K. In contrast, I-LLM suffers a 43.3% throughput reduction at 2K, increasing to 54.4% at 16K. IntAttention performs better than I-LLM but still trails **SCOPE** by 21.4%–35.5%. Overall, these results show that **SCOPE** delivers higher and more stable prefill attention throughput, with highly optimized SCNA approximation and hardware deployment for nonlinear functions.

7.3 Approximation Accuracy

7.3.1 Nonlinear Function Approximation. We evaluate approximation accuracy on 11 nonlinear functions. We report both mean squared error (MSE) and mean absolute error (MAE). Baselines include traditional methods, namely Taylor expansion and linear interpolation, and learned universal approximators, namely NN-LUT [55] and T-LUT [54]. We also include range-separated Taylor and interpolation variants, denoted Frac-T and Frac-I, which split the input into integer and fractional parts to improve precision. This optimization, however, is only applicable to exponential-family functions such as Exp and Exp2 and does not generalize to the rest of the operator set. For each function, the input grid follows the deployment range and canonicalization recipe in Table 1; for symmetric or reflected functions, the transform is applied before both approximation and error measurement. The error is computed against PyTorch floating-point outputs on the same input grid.

As shown in Table 4, SCNA achieves the best overall numerical accuracy. It reduces geomean MSE across all functions by **361×** over **NN-LUT** and **14.9×** over T-LUT. SCNA remains accurate across diverse operators, achieving MSEs of 5.02×10^{-9} on **Exp** and 1.22×10^{-6} on **Rsqrt**. NN-LUT, in contrast, incurs large errors on functions such as Sin and Rsqrt, likely because its unconstrained training does not preserve key functional structure. SCNA also matches the error level of specialized range-separated methods on Exp and Exp2, while using a single general approximation family rather than function-specific optimization. These results show that SCNA delivers both high accuracy and broad operator generality.

Table 5: Sensitivity of SCNA to low-bit quantization.

Method	W6A6				W4A4			
	Llama2-7B		Llama3-8B		Llama2-7B		Llama3-8B	
	PPL	Acc	PPL	Acc	PPL	Acc	PPL	Acc
I-LLM	5.68	N/A	6.61	N/A	10.44	N/A	21.19	N/A
OSTQuant	5.50	65.09	6.23	68.03	5.94	62.58	7.33	64.64
w/ SCNA-8	5.54	64.99	6.32	68.01	6.00	62.44	7.44	64.44
w/ SCNA-16	5.50	65.04	6.24	68.02	5.96	62.93	7.34	65.11
w/ SCNA-32	5.50	65.06	6.24	67.97	5.95	62.77	7.35	64.72
BF16 Baseline	5.47	74.43	6.14	77.61	5.47	74.43	6.14	77.61

7.3.2 LLM Accuracy and Quantization Robustness. We evaluate whether SCNA maintains model quality when exact nonlinear operators are replaced by our approximator, particularly under low-bit inference. We compare SCNA against the baseline nonlinear implementation on **OPT-6.7B**, **Llama-2-7B**, **Llama-3-8B**, **Qwen2.5-7B**, and **Qwen3-8B** in both **BF16** and **INT8**. Figure 16a reports perplexity results, while Figure 16b reports mean zero-shot accuracy. SCNA- N represents that the approximator has N neurons, where $N \in \{8, 16, 32\}$. The main deployment target is SCNA-8, while SCNA-16 and SCNA-32 serve as capacity-sensitivity points. With SCNA-8, **OPT-6.7B** slightly improves **INT8** perplexity from 10.921 to 10.875, while **Llama-2-7B** and **Llama-3-8B** increase from 5.563 to 5.681 and from 6.440 to 6.750, respectively. Results on newer models are not monotonic: **Qwen2.5-7B** increases from 8.391 to 8.706 under **INT8**, whereas **Qwen3-8B** improves from 10.054 to 9.924. Average zero-shot accuracy remains close to the baseline, with the largest SCNA-8 **INT8** drop being only 0.5 percentage points on **Qwen2.5-7B**; **Qwen3-8B** changes from 74.4% to 74.0%. Overall, these results support a conservative conclusion that SCNA introduces limited additional degradation on top of quantization, rather than losslessly preserving every metric.

7.3.3 Accuracy with low-bit dynamic quantization. **SCOPE** methods are orthogonal to the choice of quantization method. It does not constrain whether the model uses static quantization, dynamic quantization, rotations, or other transformation-based quantization techniques. To further demonstrate **SCOPE**'s compatibility with advanced dynamic quantization methods, we additionally integrate the SCNA approximator into **OSTQuant**, a recent state-of-the-art quantization method. The new results in Table 5 show that SCNA introduces only negligible degradation. With SCNA-8, the **W6A6** result changes from 5.50/65.09 to 5.54/64.99 in **WikiText** perplexity

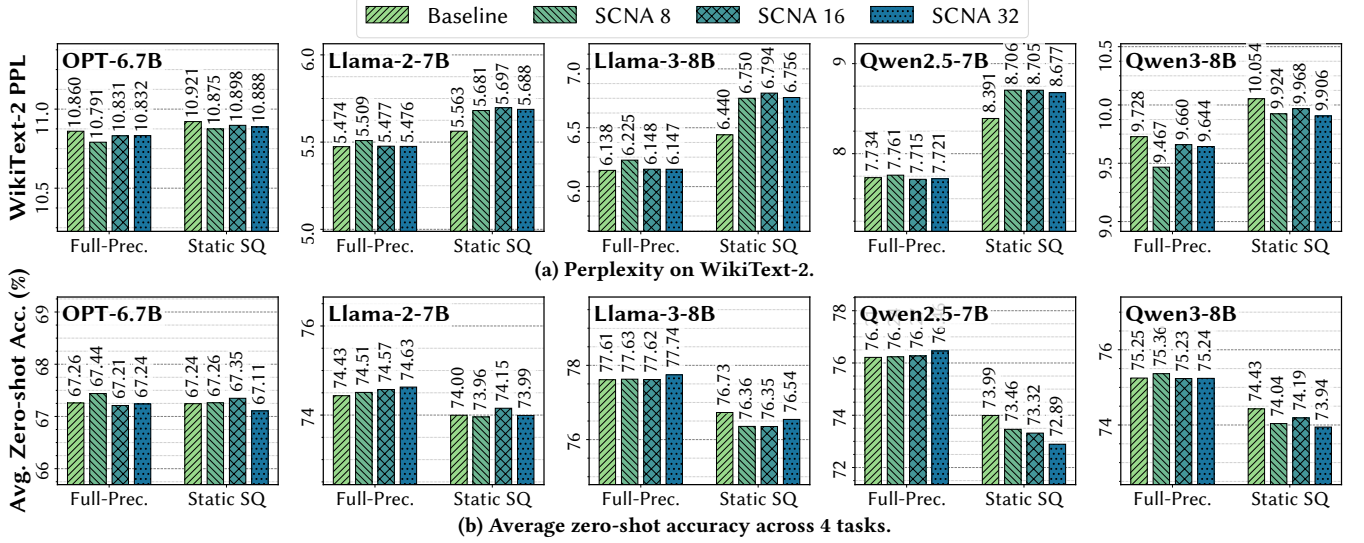


Figure 16: Impact of SCNA on LLM accuracy with full-precision and quantization.

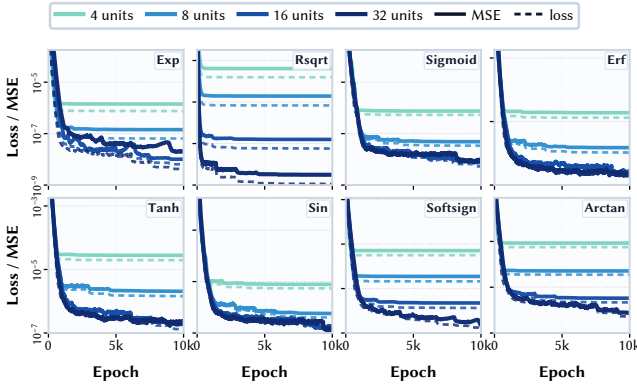


Figure 17: Loss and MSE convergence of 4, 8, 16, and 32-neuron SCNA configurations.

/ zero-shot average accuracy, and the W4A4 result changes from 5.94/62.58 to 6.00/62.44. These results confirm that SCOPE remains compatible with advanced quantization methods, while the proposed scale-fusion optimization provides an additional benefit for static quantization settings.

7.3.4 SCNA Neuron Count Scalability. We evaluate SCNA scalability with neuron count by comparing 4-, 8-, 16-, and 32-neuron variants on nine representative nonlinear functions, using training loss and final MSE. Figure 17 shows a clear scaling trend: **more neurons generally improve convergence and final accuracy. The 32-neuron model achieves the lowest error in most cases**, reducing final MSE by 97.2×–2837.8× over the 4-neuron model. The 4- and 8-neuron variants often plateau at much higher error, while the 16-neuron variant closes most of the gap and often closely tracks the 32-neuron curves. The improvement comes from higher approximation capacity for fine-grained curvature and dynamic-range variation, while shape constraints make the extra capacity effective and stable. The gains are largest for harder functions such as Rsqrt,

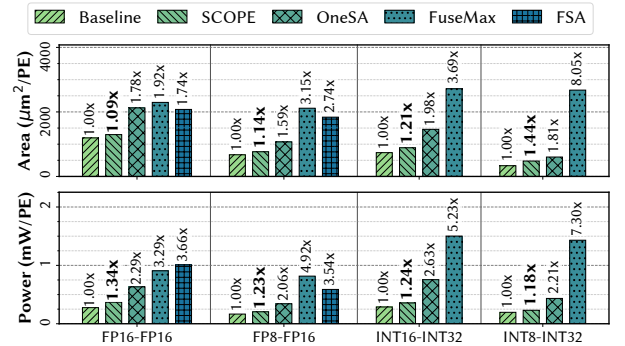


Figure 18: Area and power consumption per PE.

Softsign, and Arctan, where small models underfit, and smaller for smoother functions such as Exp and Exp2, where 16 neurons already capture most of the structure **and even yield better accuracy than 32 neurons**. Overall, SCNA scales effectively with diminishing returns near 32 neurons.

7.4 Hardware Overhead

We compare isolated per-PE area and power against prior array-fused nonlinear designs across four data-type/accumulation-type combinations. As shown in Figure 18, **SCOPE** consistently provides the lowest overhead among all augmented designs, requiring only 1.09×–1.44× area and 1.18×–1.34× power across the evaluated settings, whereas OneSA requires 1.59×–1.98× area and 2.06×–2.63× power, and FuseMax increases overhead much more substantially, up to 8.05× area and 7.30× power. This is because our array-native extension for SCNA introduces only one additional register and one lightweight ReLU unit in each augmented PE, avoiding heavy nonlinear structures such as comparators or LUTs.

We then evaluate hardware efficiency by comparing throughput-normalized overhead against both array-fused designs and standalone universal approximators under FP16 and INT32. For SCNA

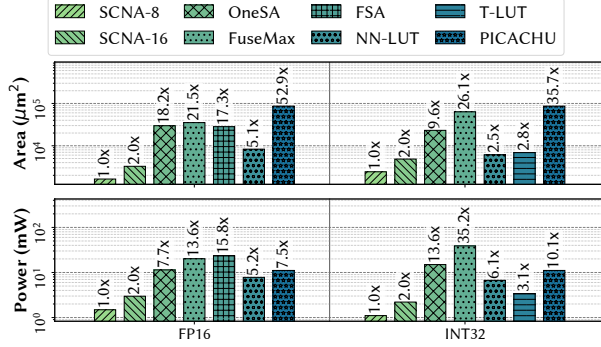


Figure 19: Normalized incremental hardware overhead.

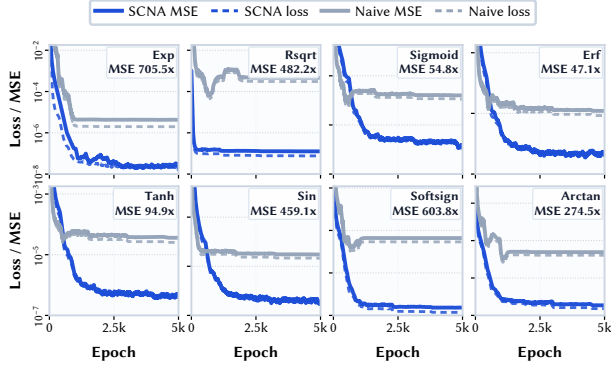


Figure 20: Loss and MSE convergence of 16-neuron SCNA and unconstrained NN baseline.

and other array-fused designs, overhead is reported incrementally over a 32×32 baseline systolic array. As shown in Figure 19, SCNA-8 uses 8 neurons for approximation, whereas SCNA-16 doubles the neuron count and correspondingly increases the hardware overhead. SCNA remains the most hardware-efficient design in every setting. In FP16, SCNA-8 uses 5.1× to 52.9× less area and 5.2× to 15.8× less power than the competing designs. In INT32, it uses 2.5× to 35.7× less area and 3.1× to 35.2× less power. On the geometric mean across all shown baseline–precision pairs, SCNA-8 is 12.8× smaller in area and 9.5× lower in power. Even SCNA-16, which doubles the SCNA-8 hardware budget, remains below every competing design shown in both area and power. Overall, these results show that SCNA delivers the same throughput with substantially lower silicon cost because its compact shared approximation datapath avoids the large tables and heavyweight function-specific hardware used by prior designs.

7.5 Ablation Analysis

7.5.1 Impact of shape constraints. We evaluate whether SCNA’s advantage primarily comes from its shape-constrained design. Under identical training settings, we compare 16-neuron SCNA against an unconstrained neural approximator on nine representative nonlinear functions. As shown by the training-loss and MSE curves in Figure 20, SCNA consistently converges to substantially lower error across all functions, reducing final MSE by 47.1×–705.5×. The gains are largest for Exp (705.5×), Rsqrt (482.2×), and Softsign (603.8×), and remain significant for Sigmoid (54.8×), Erf (47.1×),

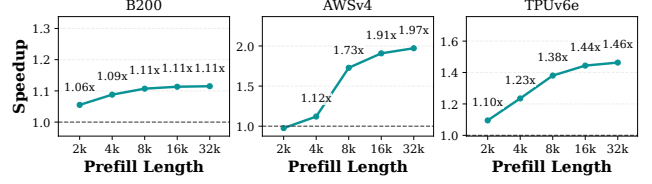


Figure 21: Impact of scale-fused nonlinear dataflow.

and Tanh (94.9×). The gain is smaller for smooth bounded S-shaped functions such as Sigmoid and Erf, which are already easier to fit even without explicit structural guidance. The reason is that shape constraints encode key functional priors, including monotonicity, convexity, symmetry, and saturation, directly into the approximator, allowing model capacity to focus on residual fitting while also regularizing optimization and avoiding the early plateau seen in the unconstrained baseline.

7.5.2 Impact of scale-fused dataflow. We quantify the end-to-end benefit of the scale-fused quantization dataflow by modeling type-conversion overhead in kernel latency using the vector throughput in Table 2. The workloads and metrics are the same as in Section 7.2. As shown in Figure 21, the scale-fused dataflow consistently outperforms the basic dataflow, and the advantage grows with sequence length: on B200, the speedup increases from 1.06× at 2K to 1.11× at 32K; on TPUv6e, it rises from 1.10× to 1.46×; and on AWSv4, it reaches 1.12× at 4K, 1.73× at 8K, 1.91× at 16K, and 1.97× at 32K. This advantage arises because scale fusion avoids extra conversion work and data movement on the critical path of the attention kernel. The benefit is larger at longer prefills and on devices such as AWSv4 and TPUv6e because conversion overhead accumulates with sequence length, and both devices have much lower vector throughput than B200. Overall, the results show that **SCOPE** achieves the lowest overhead by removing explicit conversion costs, and that this design choice becomes increasingly important as context length grows.

8 Conclusion

We present **SCOPE**, a software-hardware co-design that makes general nonlinear evaluation an array-native capability for LLM inference. As GEMMs continue to scale on systolic-array accelerators, nonlinear operators increasingly become the bottleneck, especially under quantized inference. Prior approaches reduce the cost of individual kernels, but do not simultaneously provide broad operator coverage, high fidelity, and scalable hardware efficiency. **SCOPE** addresses this gap with shape-constrained neuron approximators (SCNA), an array-native execution mechanism on lightly extended systolic arrays, and a scale-fused quantization dataflow that keeps nonlinear execution aligned with surrounding low-precision GEMMs. Across diverse nonlinear functions and five representative LLMs, **SCOPE** reduces geomean approximation MSE by 361× over NN-LUT and 14.9× over T-LUT, maintains BF16/INT8 perplexity and zero-shot accuracy with limited additional degradation, delivers up to 2.81× long-context attention speedup over FlashAttention on TPUs, achieves up to 2.69× full-prefill speedup under INT8, and lowers normalized area and power by 12.8× and 9.5×, respectively, versus prior nonlinear accelerators.

A Artifact Appendix

A.1 Abstract

The SCOPE artifact contains the software models, accuracy experiments, RTL, synthesis reports, plotting scripts, and compact measurements needed to evaluate the paper. It reproduces Tables 3–5 and Figures 13–21, including CPU-modeled accelerator performance, GPU model quality and nonlinear approximation accuracy, and area/power results extracted from native synthesis reports. A hardware-free reviewer path regenerates the submitted evidence and independently validates it. Fresh CPU, GPU, RTL, and synthesis workflows are also included.

A.2 Artifact check-list (meta-information)

- **Algorithms/programs:** The artifact includes SCOPE/SCNA approximation, LLMCompass, SCALE-Sim, OSTQuant, PyTorch, Chisel RTL, and evidence validators.
- **Compilation/transformations:** The Python and Cython programs run directly after installation. The optional RTL workflow uses sbt, Scala, and Chisel to elaborate SystemVerilog.
- **Models/data:** The model set comprises OPT-6.7B, Llama-2-7B, Llama-3-8B, Qwen2.5-7B, and Qwen3-8B. The data sets are WikiText-2, Pile validation, ARC-Easy, HellaSwag, PIQA, WinoGrande, and synthetic function samples.
- **Run-time environment:** The validated environment is 64-bit GNU/Linux with Python 3.12. Fresh GPU execution additionally uses a CUDA 12.8-compatible runtime and may use Slurm.
- **Hardware:** Bundled evidence and modeled performance run on a CPU. Fresh quality experiments use NVIDIA GPUs. Repeating synthesis requires the synthesis environment described below.
- **Execution/metrics:** Automated Make workflows report latency, throughput, speedup, perplexity, zero-shot accuracy, MSE, area, and power.
- **Output:** Outputs include CSV, JSON, and Markdown validation reports, PDF and PNG plots, tables, and generated SystemVerilog.
- **Experiments:** The artifact covers Tables 3–5, Figures 13–21, RTL regeneration, and the synthesis-report audit.
- **Resources:** Bundled review requires 2 CPU cores, 8 GB RAM, 5 GB plus dependency caches, and about 15 seconds. The fresh CPU suite requires 8 cores, 16 GB RAM, 20 GB, and about 37 minutes on dual EPYC 9654. The fresh GPU suite takes about 5 hours with up to 16 H100 GPUs, excluding setup, downloads, and scheduler queue time.
- **Preparation:** Preparing the workflow takes about 10–30 minutes with network access, excluding model approval and downloads.
- **Availability:** Zenodo archive: <https://doi.org/10.5281/zenodo.21472477>. Github repository: <https://github.com/xsun2001/SCOPE-MICRO26-AE>.

A.3 Description

A.3.1 How to access. Clone or unpack the artifact above and enter its root directory. The top-level README.md is the reviewer entry point. Every experiments/fig-* or experiments/tbl-* directory contains its own commands and expected outputs. Reproduction targets are in expected-results.

A.3.2 Hardware dependencies. Bundled evidence needs no accelerator or proprietary software. Fresh CPU experiments need roughly 8 cores and 16 GB RAM. Fresh Figure 16 and Table 5 need a CUDA GPU with at least 80 GB device memory. Figures 17 and 20 need at least 16 GB. Table 5 additionally needs about 192 GB host memory and 200 GB free storage. Independent experiments may share up to 16 GPUs.

A.3.3 Software dependencies. Required tools are GNU Make, Bash, Python 3.10 or newer (3.12 validated), and uv or venv/pip. Fresh GPU runs use the pinned requirements/accuracy.txt environment (PyTorch 2.10.0+cu128, Transformers 4.51.0, and LM Evaluation Harness 0.4.4). RTL regeneration uses OpenJDK 21 and sbt. Slurm is optional.

A.3.4 Data sets and models. CPU inputs and compact GPU evidence are included. Fresh GPU runs download the datasets listed above and require Hugging Face-format checkpoints prepared under MODEL_ROOT. Model weights and large Table 5 intermediates are not redistributed. Reviewers must obtain weights under the applicable terms.

A.4 Installation

From the artifact root, run:

```
make setup
make evidence
make validate
```

The first command creates .venv and installs dependencies. An existing environment can instead be selected by setting PYTHON environmental variable. Network access is normally needed for initial package, model, and data fetches.

A.5 Experiment workflow

- **Fast review:** make evidence && make validate regenerates compact tables/plots in ignored run directories and validates all packaged CPU/GPU results without changing tracked evidence.
- **Fresh CPU/RTL:** make reproduce-cpu JOBS=8 runs the modeled performance suite, hardware extraction, and Chisel elaboration.
- **Fresh GPU:** Reviewer can copy config/local.env.example to config/local.env for customization, including MODEL_ROOT, and setting EXECUTOR=local WORKERS=1 on an allocated GPU or EXECUTOR=slurm WORKERS=N for job arrays. Then run make reproduce-gpu.

A.6 Evaluation and expected results

A successful packaged review prints status: pass for the following checks:

- 68/68 CPU checks and 36 Table 3 groups recomputed from 720 raw H100 samples.
- 80/80 Figure 16, 20/20 Table 5, 36/36 Figure 17, and 18/18 Figure 20 comparisons.
- All 11 SCNA rows in Table 4 and zero absolute author paths.

Figures 13–15 and 21 match the reported performance and speedup targets. Figures 18–19 are regenerated from 112 filtered native synthesis report sets. Figure 16 and Table 5 both use WikiText-2

perplexity and the unweighted mean accuracy over ARC-Easy, Hel-laSwag, PIQA, and WinoGrande. Table 4 reproduces SCOPE/SCNA accuracy. Its other columns are literature reference values.

A.7 Experiment customization

Use JOBS=N for CPU workers and WORKERS=N for GPU job-array width. Make’s -j runs independent experiment targets concurrently. The product of concurrent targets and WORKERS must remain within the GPU allocation. Individual targets (for example, make fig-16 or make tbl-5) permit partial reproduction. Table 5 checkpoints may be reused through TABLE5_CHECKPOINT_SOURCE.

A.8 Methodology

Artifact reviewing and badging follow <https://www.acm.org/publications/policies/artifact-review-and-badging-current> and <https://cTuning.org/ag>.

References

- [1] Brandon Amos, Lei Xu, and J. Zico Kolter. 2017. Input convex neural networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70* (Sydney, NSW, Australia) (ICML ’17). JMLR.org, 146–155.
- [2] AWS. [n. d.]. NeuronCore-v4 Architecture; AWS Neuron Documentation — awsdocs-neuron.readthedocs-hosted.com. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/about-neuron/arch/neuron-hardware/neuron-core-v4.html>. [Accessed 07-04-2026].
- [3] AWS. 2026. Attention CTE Kernel API Reference — AWS Neuron Documentation. <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/library/api/attention-cte.html>. Accessed: 2026-04-07.
- [4] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. arXiv:1607.06450 [stat.ML] <https://arxiv.org/abs/1607.06450>
- [5] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzyniak, and Krste Asanović. 2012. Chisel: constructing hardware in a Scala embedded language. In *Proceedings of the 49th Annual Design Automation Conference* (San Francisco, California) (DAC ’12). Association for Computing Machinery, New York, NY, USA, 1216–1225. doi:10.1145/2228360.2228584
- [6] Yutong Bai, Xinyang Geng, Kartikeya Mangalam, Amir Bar, Alan L. Yuille, Trevor Darrell, Jitendra Malik, and Alexei A. Efros. 2024. Sequential Modeling Enables Scalable Learning for Large Vision Models. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22861–22872. doi:10.1109/CVPR52733.2024.02157
- [7] Yonatan Bisk, Rowan Zellers, Ronan Le bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 05 (Apr. 2020), 7432–7439. doi:10.1609/aaai.v34i05.6239
- [8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS ’20). Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.
- [9] A. Cantoni. 1971. Optimal Curve Fitting With Piecewise Linear Functions. *IEEE Trans. Comput.* C-20, 1 (1971), 59–67. doi:10.1109/T-C.1971.223082
- [10] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. arXiv:1803.05457 [cs.AI] <https://arxiv.org/abs/1803.05457>
- [11] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. arXiv:2307.08691 [cs.LG] <https://arxiv.org/abs/2307.08691>
- [12] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. GPT3.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 30318–30332. https://proceedings.neurips.cc/paper_files/paper/2022/file/c3ba4962c05c49636d4c6206a97e9c8a-Paper-Conference.pdf
- [13] Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Bin Wang, Linke Ouyang, Xilin Wei, Songyang Zhang, Haodong Duan, Maosong Cao, Wenwei Zhang, Yining Li, Hang Yan, Yang Gao, Xinyue Zhang, Wei Li, Jingwen Li, Kai Chen, Conghui He, Xingcheng Zhang, Yu Qiao, Dahua Lin, and Jiaqi Wang. 2024. InternLM-XComposer2: Mastering Free-form Text-Image Composition and Comprehension in Vision-Language Large Model. arXiv:2401.16420 [cs.CV] <https://arxiv.org/abs/2401.16420>
- [14] Xiao Dong, Xiaolei Zhu, and De Ma. 2019. Hardware Implementation of Softmax Function Based on Piecewise LUT. In *2019 IEEE International Workshop on Future Computing (IWOFC)*. 1–3. doi:10.1109/IWOFC48002.2019.9078446
- [15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. arXiv:2010.11929 [cs.CV] <https://arxiv.org/abs/2010.11929>
- [16] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. OPTQ: Accurate Quantization for Generative Pre-trained Transformers. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=tcbBPnfwxS>
- [17] Bolin Gao and Lacra Pavel. 2018. On the Properties of the Softmax Function with Application in Game Theory and Reinforcement Learning. arXiv:1704.00805 [math.OC] <https://arxiv.org/abs/1704.00805>
- [18] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027* (2020).
- [19] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonnell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2021. A framework for few-shot language model evaluation. doi:10.5281/zenodo.5371628
- [20] Soroush Ghodrati, Sean Kinzer, Hanyang Xu, Rohan Mahapatra, Yoonsung Kim, Byung Hoon Ahn, Dong Kai Wang, Lavanya Karthikeyan, Amir Yazdanbakhsh, Jongse Park, Nam Sung Kim, and Hadi Esmaeilzadeh. 2024. Tandem Processor: Grappling with Emerging Operators in Neural Networks. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (ASPLOS ’24). Association for Computing Machinery, New York, NY, USA, 1165–1182. doi:10.1145/3620665.3640365
- [21] Google. [n. d.]. TPU v6e | Google Cloud Documentation — docs.cloud.google.com. <https://docs.cloud.google.com/tpu/docs/v6e>. [Accessed 07-04-2026].
- [22] Boris Hanin and Mark Sellke. 2018. Approximating Continuous Functions by ReLU Nets of Minimal Width. arXiv:1710.11278 [stat.ML] <https://arxiv.org/abs/1710.11278>
- [23] Dan Hendrycks and Kevin Gimpel. 2023. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] <https://arxiv.org/abs/1606.08415>
- [24] Pieter-Jan Hoedt and Günter Klambauer. 2023. Principled weight initialisation for input-convex neural networks. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS ’23). Curran Associates Inc., Red Hook, NY, USA, Article 1998, 12 pages.
- [25] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. 1989. Multilayer feed-forward networks are universal approximators. *Neural Networks* 2, 5 (1989), 359–366. doi:10.1016/0893-6080(89)90020-8
- [26] Huangqi Hu, Bowen Xiao, Shixuan Sun, Jianian Yin, Zhexi Zhang, Xiang Luo, Chengquan Jiang, Weiqi Xu, Xiaoying Jia, Xin Liu, and Minyi Guo. 2025. LiquidGEMM: Hardware-Efficient W4A8 GEMM Kernel for High-Performance LLM Serving. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’25)*. Association for Computing Machinery, New York, NY, USA, 1619–1630. doi:10.1145/3712285.3759852
- [27] Xing Hu, Yuan Cheng, Dawei Yang, Zhihang Yuan, Jiangyong Yu, Chen Xu, and Sifan Zhou. 2024. I-LLM: Efficient Integer-Only Inference for Fully-Quantized Low-Bit Large Language Models. arXiv:2405.17849 [cs.LG] <https://arxiv.org/abs/2405.17849>
- [28] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture* (Orlando, FL, USA) (ISCA ’23). Association for Computing Machinery, New York, NY, USA, Article 82, 14 pages. doi:10.1145/3579371.3589350
- [29] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran

- Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snellman, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-dataloader performance analysis of a tensor processing unit. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. 1–12. doi:10.1145/3079856.3080246
- [30] Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. I-BERT: Integer-only BERT Quantization. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 5506–5518. <https://proceedings.mlr.press/v139/kim21d.html>
- [31] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. 2017. Structured Attention Networks. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=HkE0Nvqlg>
- [32] H. T. Kung. 1982. Why Systolic Architectures? *Computer* 15, 1 (Jan. 1982), 37–46. doi:10.1109/MC.1982.1653825
- [33] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [34] Jamie Li, Alexander Zhurkevich, Vedaanta Agarwalla, Seonghee Lee, and Roman Anders. 2026. Making Softmax More Efficient with NVIDIA Blackwell Ultra. <https://developer.nvidia.com/blog/making-softmax-more-efficient-with-nvidia-blackwell-ultra/>. NVIDIA Technical Blog, February 25, 2026.
- [35] Jiawei Lin, Yuanlong Li, Guokai Chen, and Thomas Bourgeat. 2025. SystolicAttention: Fusing FlashAttention within a Single Systolic Array. arXiv:2507.11331 [cs.AR] <https://arxiv.org/abs/2507.11331>
- [36] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 87–100. https://proceedings.mlsys.org/paper_files/paper/2024/file/42a452cbafa9dd64e9ba4aa95cc1ef21-Paper-Conference.pdf
- [37] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2025. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. In *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin (Eds.), Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/fbe2b2f74a2ece8070d8fb073717bda6-Paper-Conference.pdf
- [38] Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, and Liwei Wang. 2017. The Expressive Power of Neural Networks: A View from the Width. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/32cbf687880eb1674a07bf17761dd3a-Paper.pdf
- [39] Nandeeeka Nayak, Xinrui Wu, Toluwanimi O. Odemuyiwa, Michael Pellauer, Joel S. Emer, and Christopher W. Fletcher. 2024. FuseMax: Leveraging Extended Einsums to Optimize Attention Accelerator Design. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1458–1473. doi:10.1109/MICRO61859.2024.00107
- [40] NVIDIA. [n. d.]. NVIDIA DGX B200 – nvidia.com. <https://www.nvidia.com/en-us/data-center/dgx-b200/>. [Accessed 07-04-2026].
- [41] Sejun Park, Chulhee Yun, Jaeho Lee, and Jinwoo Shin. 2021. Minimum Width for Universal Approximation. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=O-XJwyoLF-k>
- [42] H C Prashanth and Madhav Rao. 2022. SOMALib: Library of Exact and Approximate Activation Functions for Hardware-efficient Neural Network Accelerators. In *2022 IEEE 40th International Conference on Computer Design (ICCD)*. 746–753. doi:10.1109/ICCD56317.2022.00114
- [43] Jiajun Qin, Tianhua Xia, Cheng Tan, Jeff Zhang, and Sai Qian Zhang. 2025. PI-CACHU: Plug-In CGRA Handling Upcoming Nonlinear Operations in LLMs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 845–861. doi:10.1145/3676641.3716013
- [44] Ritik Raj, Sarbartha Banerjee, Nikhil Chandra, Zishen Wan, Jianming Tong, Ananda Samajdhara, and Tushar Krishna. 2025. SCALE-Sim V3: a Modular Cycle-Accurate Systolic Accelerator Simulator for End-To-End System Analysis. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 186–200. doi:10.1109/ISPASS64960.2025.00026
- [45] Davor Runje and Sharath M Shankaranarayana. 2023. Constrained Monotonic Neural Networks. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 29338–29353. <https://proceedings.mlr.press/v202/runje23a.html>
- [46] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: an adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (Aug. 2021), 99–106. doi:10.1145/3474381
- [47] Tim Salimans and Diederik P. Kingma. 2016. Weight normalization: a simple reparameterization to accelerate training of deep neural networks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (Barcelona, Spain) (NIPS '16)*. Curran Associates Inc., Red Hook, NY, USA, 901–909.
- [48] Noam Shazeer. 2020. GLU Variants Improve Transformer. arXiv:2002.05202 [cs.LG] <https://arxiv.org/abs/2002.05202>
- [49] Ruiqi Sun, Yinchun Ni, Xin He, Jie Zhao, and An Zou. 2024. ONE-SA: Enabling Nonlinear Operations in Systolic Arrays For Efficient and Flexible Neural Network Inference. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. doi:10.23919/DATE58400.2024.10546535
- [50] Ruiqi Sun, Siwei Ye, Jie Zhao, Xin He, Jianzhe Lin, Yiran Li, and An Zou. 2024. NeuralMatrix: Compute the Entire Neural Networks with Linear Matrix Operations for Efficient Inference. arXiv:2305.14405 [cs.LG] <https://arxiv.org/abs/2305.14405>
- [51] Synopsys, Inc. 2026. DesignWare Intellectual Property. [Online]. Available: <https://www.synopsys.com/dw/buildingblock.php>
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
- [53] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 38087–38099. <https://proceedings.mlr.press/v202/xiao23c.html>
- [54] Yusheng Xie, Alex Noel Joseph Raj, Zhendong Hu, Shaohaoan Huang, Zhun Fan, and Mirosław Joler. 2020. A Twofold Lookup Table Architecture for Efficient Approximation of Activation Functions. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 28, 12 (2020), 2540–2550. doi:10.1109/TVLSI.2020.3015391
- [55] Joonsang Yu, Junki Park, Seongmin Park, Minsoo Kim, Sihwa Lee, Dong Hyun Lee, and Jungwook Choi. 2022. NN-LUT: neural approximation of non-linear operations for efficient transformer inference. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (San Francisco, California) (DAC '22)*. Association for Computing Machinery, New York, NY, USA, 577–582. doi:10.1145/3489517.3530505
- [56] Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. 2026. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451 [cs.CL] <https://arxiv.org/abs/2603.05451>
- [57] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Anna Korhonen, David Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, Florence, Italy, 4791–4800. doi:10.18653/v1/P19-1472
- [58] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. Curran Associates Inc., Red Hook, NY, USA.
- [59] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 1080–1096. doi:10.1109/ISCA59077.2024.00082
- [60] Wanli Zhong, Haibo Feng, Zirui Zhou, Hanyang Peng, and Shiqi Yu. 2026. IntAttention: A Fully Integer Attention Pipeline for Efficient Edge Inference. In *Ninth Conference on Machine Learning and Systems*. <https://openreview.net/forum?id=CPCRITWAAp>